

Proyecto Final: Ingeniería Web y Móvil

1. Información

- Deben usar para el frontend el framework Ionic con React y para el backend Node.js, Flask, Django, FastAPI, etc.
- El software debería ser implementado en su totalidad por el grupo de trabajo.
- El problema a resolver deber ser propuesto y modelado por los integrantes del grupo.
- El grupo deberá estar conformado máximo de 4 estudiantes. Los estudiantes deberán inscribir sus proyectos en el siguiente link Formulario de Google, el cual tendrá como fecha **límite 22 de Agosto del 2026**
- El proyecto deberá ser gestionado a través de GitHub, manteniendo un repositorio público con el código fuente del proyecto. También deberán hacer uso de **readme.md** usando adecuadamente Markdown (md)

2. Restricciones

- Los proyectos no se pueden repetir más de 3 veces. En caso de que exista un tema igual o similar, uno de los grupos deberá cambiar su tema, dándose prioridad al grupo que inscribió primero la propuesta.
- El frontend deberá ser desarrollado utilizando el framework Ionic con React.
- El backend deberá ser desarrollado aplicando APIs RESTful con conexión a base de datos relacional, usando Node.js con express o Flask (Python).
- Todos los integrantes del grupo deben pertenecer al mismo paralelo.

- El repositorio del proyecto en Github y el acceso a Figma deberán ser públicos. Si no es público tendrá una penalización de un 1.0 tomando en escala de 1.0 a 7.0

3. Gestión del Proyecto

El proyecto deberá gestionarse mediante GitHub. Para ello, cada equipo deberá crear y mantener un repositorio público, que contenga el código fuente actualizado, los recursos actualizados y la documentación asociada al desarrollo del proyecto. El proyecto deberá evidenciar un proceso de desarrollo continuo y colaborativo. No se considerará suficiente cargar la totalidad del proyecto en una única entrega o realizar todos los **commits** para cada entrega parcial.

3.1. Archivo README.md

El repositorio deberá incluir un archivo **README.md** correctamente estructurado y redactada utilizando sintaxis Markdown. Este archivo constituirá a la documentación principal del proyecto y deberá contener, los siguientes elementos:

- nombre del proyecto;
- identificación de los integrantes del equipo;
- distribución de responsabilidades;
- descripción general del sistema;
- problema o necesidad que aborda;
- objetivos del proyecto;
- principales funcionalidades;
- tecnologías y herramientas utilizadas;
- instrucciones de instalación y configuración;

- instrucciones de ejecución y uso;
- imágenes, diagramas, hipervínculos, vídeos deben ir dentro del mismo documento usando markdown

3.2. Uso y contenido de las ramas

Cada equipo deberá organizar el desarrollo del proyecto utilizando las siguientes ramas:

- **frontend**: deberá contener el código correspondiente a la interfaz de usuario y los componentes visuales del sistema. Además, deberá incluir la documentación de los requerimientos funcionales y no funcionales del proyecto, incorporada directamente en un archivo Markdown, preferentemente `README.md`.
- **backend**: deberá contener la lógica del servidor, las API, la conexión con la base de datos y los demás componentes relacionados con el procesamiento de la información. Asimismo, deberá incluir un archivo con extensión `.sql` que permita crear la estructura de la base de datos utilizada por el sistema.

El archivo `.sql` deberá contener, como mínimo, la creación de la base de datos, las tablas, sus atributos, claves primarias, claves foráneas y relaciones correspondientes. Cuando sea necesario para demostrar el funcionamiento del sistema, también podrá incorporar datos iniciales o de prueba. La documentación de los requerimientos incluida en la rama **frontend** deberá presentar cada requerimiento mediante un identificador, nombre y descripción. Los requerimientos funcionales deberán especificar las acciones que podrá realizar el sistema, mientras que los requerimientos no funcionales deberán describir condiciones relacionadas con aspectos como usabilidad, seguridad, rendimiento, accesibilidad y compatibilidad.

3.3. Registro del repositorio

A través del Aula Virtual de la asignatura se compartirá una planilla en formato Excel para la inscripción de los equipos de trabajo. Cada equipo

deberá registrar en dicha planilla la información solicitada, incluyendo:

- nombre o identificación del equipo;
- nombre completo de cada integrante;
- enlace al repositorio público de GitHub del proyecto.

El enlace registrado deberá permitir el acceso directo al repositorio y mantenerse actualizado y disponible durante todo el desarrollo del proyecto. Será responsabilidad de cada equipo verificar que el enlace sea correcto y que el repositorio tenga configurada la visibilidad pública. La entrega de un enlace incorrecto, privado o sin acceso podrá impedir la revisión y evaluación del proyecto.

4. Entregas Parciales

1. Entrega Parcial 1: Diseño y Estructura inicial

Objetivo: Definir el diseño de la aplicación y construir la estructura del frontend en **Ionic+ React**.

- **EP 1.1:** Definición de al menos 7 requerimientos funcionales y al menos 5 no funcionales (rendimiento, seguridad, usabilidad). Estas funcionalidades no pueden ser repetitivas. Estas funcionalidades están fuera de inicio de sesión o registrarse ya que deben estar inmersas en la propuesta. Dentro de las funcionalidades deben considerar dos tipos de roles (ejemplo: usuario y admin).
- **EP 1.2: Justificación del problema y caracterización de los usuarios objetivo.** El equipo deberá describir y justificar el problema que abordará la aplicación, explicando su contexto, relevancia y las consecuencias asociadas a su falta de solución. Debido a que durante esta etapa no se realizará contacto directo con usuarios, la caracterización de los usuarios objetivo deberá elaborarse mediante investigación documental y análisis de soluciones existentes. Para ello, podrán utilizarse fuentes académicas, informes, estadísticas públicas, sitios institucionales, normativas, casos de estudio y aplicaciones similares. La entrega deberá incluir:

- definición del grupo o grupos de usuarios a los cuales estará dirigida la aplicación;
- descripción general de sus características, necesidades y posibles dificultades;
- contexto en el que utilizarían la aplicación;
- objetivos o tareas que realizarían dentro del sistema;
- nivel estimado de experiencia tecnológica;
- necesidades de accesibilidad, seguridad o privacidad, cuando corresponda;
- identificación de los roles considerados en el sistema;
- elaboración de, al menos, dos perfiles de usuario hipotéticos o *proto-personas*;
- identificación de los supuestos utilizados para construir dichos perfiles.

Cada *proto-persona* deberá incluir, como mínimo:

- tipo de usuario o rol;
- características generales;
- necesidades principales;
- objetivos de uso;
- dificultades o puntos de frustración;
- funcionalidades de la aplicación que utilizaría;
- dispositivo y contexto probable de acceso.

Los perfiles contruidos no deberán presentarse como resultados obtenidos de usuarios reales. El equipo deberá indicar claramente que corresponden a una caracterización preliminar basada en fuentes secundarias y supuestos razonados.

- **EP 1.3:** Bocetos de UI/UX y prototipo en Figma de al menos 7 mockups o pantallas distintas, cada una correspondiente a una funcionalidad previamente definida en los requerimientos del proyecto. Cada pantalla deberá presentar un diseño diferenciado, coherente con el flujo de navegación y la jerarquía de información. Las interfaces deberán ser prototipadas considerando explícitamente: versión móvil y web. El diseño deberá evidenciar distribución de contenido, componentes de navegación (por ejemplo: menú lateral en web, barra inferior en móvil), jerarquía visual

y densidad de la información. Se deberá considerar incluir formulario de registro e inicio de sesión.

Los bocetos, *mockups* y prototipos deberán ser elaborados manualmente por los integrantes del equipo mediante las herramientas de diseño disponibles en Figma. No se permitirá utilizar el asistente de inteligencia artificial de Figma ni otras herramientas de IA generativa para crear automáticamente las pantallas, componentes, estilos, estructuras de navegación o prototipos solicitados. El equipo deberá evidenciar la construcción propia del diseño, incluyendo la organización de los elementos, la selección de componentes, la jerarquía visual, la adaptación entre las versiones móvil y web, y la configuración de las interacciones del prototipo. Se permitirá utilizar las funciones convencionales de Figma, tales como formas, marcos, componentes, estilos, restricciones, *Auto Layout* y conexiones de prototipado. También podrán emplearse recursos gráficos externos, siempre que se encuentren correctamente referenciados y no sustituyan el trabajo de diseño realizado por el equipo.

- El formulario de registro deberá incluir los campos necesarios para identificar y registrar adecuadamente a los usuarios de la aplicación. La selección de estos campos deberá ajustarse al problema abordado, a las características de los usuarios objetivo y a los requerimientos definidos por cada equipo. Los equipos deberán justificar brevemente la información solicitada, evitando incorporar datos personales que no sean necesarios para el funcionamiento de la aplicación. El formulario podrá considerar, por ejemplo, nombre de usuario, correo electrónico, contraseña, confirmación de contraseña, aceptación de términos y condiciones u otros campos pertinentes para la propuesta. Los formularios de inicio de sesión y registro deberán representar visualmente:
 - campos obligatorios y opcionales;
 - formato esperado de los datos;
 - validaciones de entrada;
 - mensajes de error claros;
 - retroalimentación ante el envío del formulario;
 - condiciones de seguridad aplicables a las contraseñas;

- una experiencia de uso coherente con los usuarios objetivo.
- **EP 1.4:** Definición de Arquitectura de Navegación y Experiencia del Usuario. El equipo deberá definir la arquitectura de navegación de la aplicación, describiendo la estructura de rutas, jerarquía de vistas, y flujo de interacción entre pantallas. La entrega deberá incluir: (a) Rutas principales y secundarias; (b) Relaciones jerárquicas entre vistas; (c) Flujo de navegación entre funcionalidades; (d) diferenciación de acceso según roles (por ejemplo: usuario /administrador); (e) flujo de principales tareas (task flow), (f) puntos críticos de interacción; (g) coherencia de experiencia entre dispositivos; (h) breve justificación técnica de las decisiones adoptadas, considerando usabilidad, eficiencia de interacción, claridad estructural y escalabilidad de la arquitectura frontend.
- **EP 1.5:** Creación del proyecto en Ionic con React, considerando: (a) Uso de react router; (b) Rutas públicas y rutas protegidas; (c) Redirecciones (ejemplo: login obligatorio); (d) Estructura modular de vistas.

Los equipos podrán apoyarse en asistentes de inteligencia artificial, complementos de Figma o herramientas de generación de código para orientar la implementación de las interfaces. Sin embargo, deberán considerar que el código generado automáticamente puede corresponder a React convencional y no necesariamente ser compatible con la estructura, los componentes y los patrones de navegación propios de Ionic. Por lo tanto, el equipo será responsable de revisar, comprender, corregir y adaptar el código generado para integrarlo adecuadamente en una aplicación desarrollada con Ionic y React. La implementación final deberá:

- utilizar componentes propios de Ionic, tales como `IonPage`, `IonHeader`, `IonContent`, `IonMenu`, `IonTabs`, `IonInput` e `IonButton`;
- integrarse correctamente con la estructura de rutas de la aplicación;
- mantener una organización modular y comprensible del código;
- funcionar adecuadamente en versiones móvil y web;
- respetar el diseño y los flujos definidos en el prototipo de Figma;

- evitar código innecesario, duplicado o que el equipo no pueda explicar;
- cumplir con los requerimientos funcionales y no funcionales definidos para el proyecto.

El uso de estas herramientas se considerará solamente un mecanismo de apoyo. No se aceptará como implementación final la copia directa de código generado automáticamente sin revisión, adaptación, integración y validación por parte del equipo.

- **EP 1.6:** Diseño de pantallas principales e incorporando una estructura de navegación funcional y coherente con la arquitectura previamente definida en ionic-react (al menos 4). Uso de componentes propios de Ionic (IonPage, IonHeader, IonContent, IonTabs, IonMenu, etc). Separación estructural del código en carpetas (pages, components, routes, services).

Entrega:

- Repositorio GitHub/GitLab con el código del frontend
- Prototipo UI/UX en Figma.
- Aplicación navegable con diseño básico realizado en Ionic + React.
- Readme.md que incluya la documentación técnica.

2. Entrega Parcial 2: Integración frontend + backend y Autenticación

Objetivo: Implementar el backend en Node.js con Express, Flask u otro framework backend, y conectarlo con el frontend y agregar autenticación.

- **EP 2.1:** Creación del servidor en Node.js con Express o Flask
- **EP 2.2:** Configuración y modelado de la base de datos relacional. Deberán incluir el diseño modelo relacional y la implementación en base de datos usando PostgreSQL/ MySQL, uso de ORM (opcional) y validación de integridad.
- **EP 2.3:** Desarrollo de API REST con endpoints básicos (GET, POST, PUT/PATCH, y DELETE). Manejo adecuado de códigos HTTP y respuestas en formato JSON estructurado.

- **EP 2.4:** Consumo de la API REST desde Ionic con React utilizando fetch o Axios, implementando manejo de errores, interceptores y gestión de tokens JWT.
- **EP 2.5:** Implementación de autenticación con JWT deberá incluir (a) formulario de registro e inicio de sesión; (b) rutas protegidas en frontend; (c) generación y validación de JWT; (d) diferenciación por roles.
- **EP 2.6:** Validación de usuarios y manejo de sesiones deberá incluir: (a) validación de inputs ; (b) Hash de contraseñas con bcrypt; (c) Manejo seguro de credenciales; (d) Protección básica contra inyección SQL.
- **EP 2.7:** Pruebas funcionales, considerando (a) Pruebas en Postman o Insomnia; (b) Documentación de endpoints; (c) Evidencia de pruebas.

Entrega:

- Backend funcional con API REST
- Autenticación y manejo de usuarios
- Integración frontend + backend con pruebas en Postman o Insomnia.
- Repositorio actualizado o crear un nuevo repositorio para backend y otro para front-end.

3. **Entrega Final:** Funcionalidades avanzadas, optimizar rendimiento y preparar aplicación para su despliegue.

Objetivo: Agregar funcionalidades avanzadas, optimizar rendimiento y preparar la aplicación para su despliegue.

- **EF 1:** Implementación de funcionalidades completas (CRUD, notificaciones, almacenamiento local) de acuerdo a los requerimientos funcionales y no funcionales definidos.
- **EF 2:** Mejoras en UI/UX y optimización de rendimiento.
- **EF 3:** Implementación de seguridad avanzada en API: protección contra inyección SQL/XSS, implementación CORS seguro, y encriptación de datos sensibles con bcrypt.

- **EF 4:** Optimización de consultas y respuesta eficiente.
- **EF 5:** Integración con algún servicio externo (AWS o APIs de terceros).
- **EF 6:** Despliegue con docker
 - Creación de Dockerfile para backend y frontend y docker compose.
 - Configuración de docker-compose para orquestación de servicios (API, base de datos, frontend).
 - Pruebas de despliegue local con Docker.

Entrega:

- Aplicación con CRUD completo, UI/UX optimizado y seguridad web aplicada.
- Backend seguro con validaciones y cifrado de datos.
- Documentación técnica del proyecto integrado en Markdown.

5. Tecnologías y Herramientas

Frontend:

- Ionic, React, TypeScript.
- Ionic Native y Capacitor.
- CSS con Tailwind o Bootstrap.

Backend:

- Node.js con Express.js
- Flask con Python
- Bases de datos : PostgreSQL, MongoDB o MySQL
- Autenticación: JWT, OAuth.

- Cloud Services: AWS, Firebase.

Desarrollo y Despliegue:

- Git y Github/ GitLab
- Postman o Insomnia para pruebas de API
- Docker para entornos de desarrollo.

6. Rúbrica de Evaluación de la Entrega Parcial 1

Propósito: evaluar la definición del problema, los requerimientos, el diseño UI/UX, la arquitectura de navegación y la implementación inicial del frontend en Ionic con React.

Puntaje total: 100 puntos.

6.1. Distribución de puntajes de EP1

Criterio	Descripción general	Puntaje
EP 1.1	Requerimientos funcionales, no funcionales y roles.	15 pts
EP 1.2	Justificación del problema y caracterización de usuarios mediante fuentes secundarias.	10 pts
EP 1.3	Diseño UI/UX y prototipo manual en Figma.	25 pts
EP 1.4	Arquitectura de navegación y experiencia de usuario.	15 pts
EP 1.5	Configuración y estructuración del frontend en Ionic con React.	15 pts
EP 1.6	Pantallas principales implementadas en Ionic con React.	15 pts
Documentación	Rama <code>frontend</code> , <code>README.md</code> , Markdown y gestión en GitHub.	5 pts
Total		100 pts

Cuadro 1: Distribución de puntajes de la Entrega Parcial 1.

6.2. Rúbrica analítica de EP1

Cuadro 2: Rúbrica analítica de la Entrega Parcial 1.

Criterio	Pts.	Excelente	Bueno	Suficiente	Insuficiente
EP 1.1 Requerimientos y roles	15	Define al menos siete requerimientos funcionales claros, pertinentes, verificables y no repetitivos, además de cinco requerimientos no funcionales. Las funcionalidades están vinculadas con el problema, no contabilizan registro ni autenticación y diferencian claramente al menos dos roles. 13–15 pts.	Presenta la cantidad solicitada, pero algunos requerimientos requieren mayor precisión, medición o diferenciación. Los roles están definidos, aunque su relación con ciertas funciones puede mejorar. 10–12 pts.	Presenta requerimientos generales, poco medibles o parcialmente repetidos. La vinculación con el problema y la diferenciación por roles son limitadas. 7–9 pts.	Presenta menos de siete requerimientos funcionales o menos de cinco no funcionales; predominan funciones de registro o autenticación, o no se evidencian roles diferenciados. 0–6 pts.
EP 1.2 Problema y usuarios objetivo	10	Justifica con claridad el problema, su contexto, relevancia y necesidad. Caracteriza a los usuarios con fuentes secundarias pertinentes, incluye al menos dos <i>proto-personas</i> y diferencia expresamente la evidencia de los supuestos preliminares. 9–10 pts.	Presenta una justificación adecuada y una caracterización respaldada, aunque algunas necesidades, fuentes, contextos o supuestos requieren mayor profundidad. 7–8 pts.	Explica el problema de manera general y presenta una caracterización básica, con pocas fuentes o <i>proto-personas</i> incompletas. 5–6 pts.	La justificación es confusa o débil; no identifica adecuadamente a los usuarios, no utiliza fuentes o presenta supuestos como si fueran resultados de usuarios reales. 0–4 pts.
EP 1.3 UI/UX y Figma	25	Presenta al menos siete pantallas asociadas a funcionalidades, además de inicio de sesión y registro, con versiones móvil y web. Existe coherencia visual, navegación prototipada, jerarquía, densidad adecuada, validaciones y adaptación al dispositivo. Los diseños evidencian construcción manual en Figma, sin generación automática de pantallas mediante su asistente de IA. 22–25 pts.	Cumple la cantidad y las versiones solicitadas, pero algunas pantallas presentan menor diferenciación, profundidad de prototipado, adaptación o validación. La elaboración manual es mayormente evidente. 18–21 pts.	El prototipo es básico, algunas vistas no se conectan claramente con los requerimientos o una de las versiones está incompleta. La navegación, las validaciones o la evidencia de construcción manual son limitadas. 12–17 pts.	Presenta menos de las pantallas solicitadas, no incluye versión móvil y web, carece de coherencia o flujo, omite login o registro, o las pantallas fueron generadas automáticamente sin evidencia de diseño manual del equipo. 0–11 pts.

Criterio	Pts.	Excelente	Bueno	Suficiente	Insuficiente
EP 1.4 Arquitectura de navegación	15	Define rutas principales y secundarias, jerarquía, flujos, acceso por roles, <i>task flows</i> , puntos críticos y coherencia entre dispositivos. Justifica las decisiones por usabilidad, eficiencia, claridad y escalabilidad. 13–15 pts.	La arquitectura es clara y coherente, aunque algunos <i>task flows</i> , puntos críticos, roles o fundamentos técnicos requieren mayor detalle. 10–12 pts.	Describe la navegación de manera general, con omisiones en rutas secundarias, jerarquía, roles o flujos. La justificación es superficial. 7–9 pts.	No presenta una arquitectura clara o las rutas, roles y flujos son inexistentes o incoherentes con el proyecto. 0–6 pts.
EP 1.5 Estructura Ionic con React	15	El proyecto está correctamente configurado en Ionic con React, utiliza React Router, rutas públicas y protegidas, redirecciones, separación por roles y una estructura modular. Se ejecuta siguiendo el README.md. 13–15 pts.	El proyecto funciona y utiliza rutas, pero algunas protecciones, redirecciones, roles o elementos de modularidad están incompletos. Requiere ajustes menores para ejecutarse. 10–12 pts.	La configuración de rutas es básica o parcial y la estructura modular es limitada. La separación entre rutas públicas y protegidas no es clara. 7–9 pts.	No existe un proyecto funcional en Ionic con React, no puede ejecutarse o no utiliza una estructura reconocible de rutas. 0–6 pts.
EP 1.6 Interfaces implementadas	15	Implementa al menos cuatro pantallas coherentes con Figma y la arquitectura definida. Usa correctamente componentes Ionic, navegación funcional, diseño adaptable y separación en pages , components , routes y services . 13–15 pts.	Implementa al menos cuatro pantallas y la navegación funciona en la mayoría de los casos. Usa componentes Ionic, con problemas menores de estructura, reutilización o adaptación. 10–12 pts.	Implementa algunas pantallas, pero la navegación es parcial, el uso de Ionic es básico o la organización del código requiere mejoras importantes. 7–9 pts.	Presenta menos de cuatro pantallas funcionales, no evidencia uso adecuado de componentes Ionic o utiliza código generado sin adaptarlo ni poder explicar su integración. 0–6 pts.
Documentación y GitHub	5	Repositorio accesible y actualizado; rama frontend correctamente utilizada; README.md completo con requisitos, instalación, ejecución, integrantes, imágenes e hipervínculos integrados mediante Markdown; historial de contribuciones claro. 5 pts.	Repositorio disponible y documentación presente, pero con instrucciones, recursos Markdown o evidencia de gestión parcialmente incompletos. 4 pts.	Repositorio accesible, pero con documentación mínima, organización débil o dificultades para ejecutar y revisar el proyecto. 3 pts.	Repositorio inaccesible o incompleto, sin README.md adecuado, sin rama frontend reconocible o sin evidencia de participación. 0–2 pts.

6.3. Penalizaciones aplicables a EP1

Situación	Consecuencia
Repositorio sin acceso para el profesor(a) o el ayudante.	Penalización de un punto en escala de 1 a 7.
No se registra la URL del repositorio en la planilla Excel del Aula Virtual.	−5 pts.
El proyecto no puede ejecutarse por falta de dependencias o instrucciones.	−5 a −10 pts.
No existe prototipo en Figma.	Puntaje máximo de EP1: 75 pts.
No existe proyecto en Ionic con React.	Puntaje máximo de EP1: 70 pts.
No se evidencia participación del grupo en GitHub.	−5 pts.
Código generado o copiado sin revisión, adaptación ni evidencia de comprensión.	Según el reglamento académico.
Entrega fuera de plazo.	Según el reglamento del curso.

Cuadro 3: Penalizaciones aplicables a la Entrega Parcial 1.

7. Rúbrica de Evaluación de la Entrega Parcial 2

Propósito: verificar que el equipo implementó un backend funcional, conectado a una base de datos relacional, con API REST, autenticación JWT, seguridad básica, integración con Ionic y React, y evidencia de pruebas.

Puntaje total: 100 puntos.

7.1. Distribución de puntajes de EP2

Criterio	Descripción general	Puntaje
EP 2.1	Servidor backend funcional.	10 pts
EP 2.2	Base de datos relacional y archivo .sql.	15 pts
EP 2.3	API REST y endpoints CRUD.	20 pts
EP 2.4	Autenticación con JWT y roles.	15 pts
EP 2.5	Seguridad y validación de usuarios.	15 pts
EP 2.6	Integración entre Ionic con React y el backend.	10 pts
EP 2.7	Pruebas funcionales y documentación de endpoints.	10 pts
Documentación	Rama backend, README.md y evidencia de gestión.	5 pts
Total		100 pts

Cuadro 4: Distribución de puntajes de la Entrega Parcial 2.

7.2. Rúbrica analítica de EP2

Cuadro 5: Rúbrica analítica de la Entrega Parcial 2.

Criterio	Pts.	Excelente	Bueno	Suficiente	Insuficiente
EP 2.1 Servidor backend	10	Servidor implementado en un framework autorizado. Se ejecuta correctamente y presenta configuración y separación clara entre rutas, controladores y servicios. 9–10 pts.	Servidor funcional, pero con organización parcial o dependencias y configuración insuficientemente documentadas. 7–8 pts.	El servidor inicia, pero presenta errores frecuentes, estructura débil o rutas muy limitadas. 5–6 pts.	No existe un servidor funcional o la solución no corresponde a un backend real. 0–4 pts.
EP 2.2 Base de datos relacional	15	Presenta un modelo relacional coherente, tablas, claves primarias y foráneas, relaciones, restricciones e integridad. Implementa PostgreSQL o MySQL, existe conexión real con el backend y la rama backend incluye un archivo .sql reproducible. 13–15 pts.	La base de datos funciona, pero algunas relaciones, restricciones, instrucciones o elementos del archivo .sql están incompletos. 10–12 pts.	Existe una base de datos mínima y conexión, pero el modelo refleja parcialmente el problema, presenta integridad débil o el archivo .sql no permite una reproducción completa. 7–9 pts.	No existe una base de datos relacional funcional, no hay conexión real, se utilizan datos estáticos o no se entrega el archivo .sql . 0–6 pts.
EP 2.3 API REST y CRUD	20	Implementa endpoints GET, POST, PUT/PATCH y DELETE asociados al modelo del proyecto. Utiliza códigos HTTP adecuados, validación y respuestas JSON consistentes y estructuradas. 18–20 pts.	Implementa la mayoría de los endpoints, con inconsistencias menores en códigos HTTP, nombres, validación o formato JSON. 14–17 pts.	El CRUD es parcial; faltan métodos relevantes o las respuestas y códigos no son consistentes. 10–13 pts.	Los endpoints no funcionan, no siguen principios REST o solo existen rutas simuladas. 0–9 pts.
EP 2.4 JWT y roles	15	Incluye registro, inicio de sesión, generación y validación de JWT, expiración, rutas protegidas y autorización diferenciada por roles. 13–15 pts.	JWT funciona, pero la protección de algunas rutas, expiración, validación o diferenciación por roles está incompleta. 10–12 pts.	La autenticación es parcial: permite iniciar sesión, pero no protege de forma adecuada los recursos o no aplica correctamente los roles. 7–9 pts.	No implementa JWT o la autenticación y autorización no son funcionales. 0–6 pts.
EP 2.5 Seguridad y validación	15	Valida entradas, utiliza bcrypt, protege credenciales, maneja errores sin revelar información sensible y previene inyección SQL mediante consultas parametrizadas u ORM. No expone secretos en el repositorio. 13–15 pts.	Incluye bcrypt y validaciones relevantes, pero faltan algunos controles, manejo robusto de errores o protección uniforme en todas las rutas. 10–12 pts.	Presenta validaciones mínimas y seguridad aplicada solo en algunas rutas o de manera inconsistente. 7–9 pts.	Almacena contraseñas sin hash, no valida datos, expone credenciales o información sensible, o utiliza consultas vulnerables. 0–6 pts.

Criterio	Pts.	Excelente	Bueno	Suficiente	Insuficiente
EP 2.6 Integración frontend-backend	10	El frontend Ionic con React consume la API mediante <code>fetch</code> o Axios, gestiona errores, sesiones y JWT mediante servicios, interceptores o una lógica organizada. Los flujos principales funcionan con datos reales. 9–10 pts.	La conexión funciona en la mayoría de las rutas, pero el manejo de errores, tokens o sesiones es parcial. 7–8 pts.	La integración es limitada, se conecta a pocas rutas o mantiene datos estáticos en funcionalidades relevantes. 5–6 pts.	No existe integración real entre el frontend y el backend. 0–4 pts.
EP 2.7 Pruebas y endpoints	10	Presenta una colección o evidencia completa en Postman o Insomnia y documenta endpoints, métodos, parámetros, autenticación, respuestas esperadas y casos de error. 9–10 pts.	Incluye pruebas funcionales, pero falta evidencia completa o documentación clara de algunos endpoints o errores. 7–8 pts.	Las pruebas son incompletas, cubren pocos casos o no explican adecuadamente las respuestas y errores. 5–6 pts.	No presenta pruebas ni documentación de endpoints. 0–4 pts.
Documentación y GitHub	5	Repositorio actualizado, ramas frontend y backend correctamente utilizadas, commits anteriores al plazo y README.md con instalación, ejecución, variables de entorno, integrantes y evidencia de gestión. 5 pts.	Repositorio disponible, pero el README.md , el historial o la organización por ramas están parcialmente completos. 4 pts.	Repositorio poco organizado, documentación mínima o instrucciones insuficientes para reproducir la aplicación. 3 pts.	No existe acceso claro al repositorio, no se puede ejecutar el proyecto o no se evidencian las ramas y archivos exigidos. 0–2 pts.

7.3. Penalizaciones aplicables a EP2

Situación	Consecuencia
Repositorio sin acceso para el profesor(a) o el ayudante.	Penalización de un punto en escala de 1 a 7.
No se registra la URL del repositorio en la planilla Excel del Aula Virtual.	−5 pts.
El backend o la aplicación integrada no puede ejecutarse por falta de instrucciones o dependencias.	−5 a −10 pts.
No existe servidor backend funcional.	Puntaje máximo de EP2: 60 pts.
No existe conexión real con una base de datos relacional.	Puntaje máximo de EP2: 70 pts.
No existe integración real entre frontend y backend.	Puntaje máximo de EP2: 80 pts.
No se entrega el archivo <code>.sql</code> en la rama <code>backend</code> .	El criterio EP 2.2 no podrá alcanzar nivel Excelente.
Entrega fuera de plazo.	Según el reglamento del curso.

Cuadro 6: Penalizaciones aplicables a la Entrega Parcial 2.

8. Distribución general de puntajes Entrega Final

Criterio de evaluación	Puntaje
EF1. Funcionalidades completas e integración funcional de la aplicación	20 pts
EF2. Mejoras de UI/UX y optimización del rendimiento	15 pts
EF3. Seguridad avanzada en la API y manejo seguro de datos	15 pts
EF4. Optimización de consultas y eficiencia de respuesta	10 pts
EF5. Integración con servicio externo o API de terceros	10 pts
EF6. Despliegue local mediante Docker y docker-compose	15 pts
Presentación y defensa del proyecto	15 pts
Total	100 pts

Cuadro 7: Distribución de puntajes de la Entrega Final.

9. Rúbrica analítica

Cuadro 8: Rúbrica analítica para la evaluación de la Entrega Final.

Criterio	Puntaje	Excelente	Bueno	Básico	Insuficiente
EF1. Funcionalidades completas e integración funcional de la aplicación	20 pts	Implementa de forma completa y funcional los requerimientos definidos. Incluye CRUD completo, autenticación, diferenciación por roles, notificaciones, almacenamiento local y flujo integrado entre frontend y backend. La aplicación es estable y ejecutable.	Implementa la mayoría de las funcionalidades principales. El CRUD funciona en la mayor parte de los casos y existe integración frontend-backend, aunque presenta errores menores o funcionalidades secundarias incompletas.	Implementa parcialmente las funcionalidades. El CRUD o la integración presentan fallas relevantes. Algunas funciones dependen de datos simulados, están incompletas o no se relacionan claramente con los requerimientos originales.	No implementa las funcionalidades centrales, la aplicación no es ejecutable o no existe integración real entre frontend y backend.
EF2. Mejoras de UI/UX y optimización del rendimiento	15 pts	Presenta una interfaz clara, consistente, responsiva y coherente entre versión móvil y web. Usa adecuadamente componentes Ionic, navegación fluida, validaciones visuales y tiempos de carga adecuados.	La interfaz es funcional y mayormente consistente. Existen mejoras de diseño y navegación, aunque pueden persistir problemas menores de responsividad, densidad visual o rendimiento.	La interfaz permite usar la aplicación, pero presenta problemas evidentes de usabilidad, navegación, coherencia visual o rendimiento. El diseño no siempre respeta el flujo definido.	La interfaz es confusa, incompleta o difícil de usar. No evidencia mejoras respecto a entregas anteriores o presenta fallas graves de visualización y rendimiento.
EF3. Seguridad avanzada en la API y manejo seguro de datos	15 pts	Implementa validaciones robustas, protección contra inyección SQL y XSS, CORS configurado de forma segura, manejo correcto de JWT, rutas protegidas, hash de contraseñas con bcrypt y manejo seguro de credenciales.	Implementa mecanismos de seguridad principales, incluyendo JWT, bcrypt y validaciones. Presenta detalles menores pendientes en CORS, sanitización de entradas o control de errores.	Incluye seguridad básica, pero incompleta. Hay validaciones parciales, manejo débil de errores, configuración insegura o protección insuficiente frente a entradas maliciosas.	No implementa medidas de seguridad relevantes, almacena credenciales de forma insegura o permite acceso no autorizado a rutas o datos sensibles.
EF4. Optimización de consultas y eficiencia de respuesta	10 pts	Las consultas a la base de datos están bien estructuradas, usan filtros, relaciones e índices cuando corresponde. Las respuestas son eficientes, consistentes y no exponen datos innecesarios.	Las consultas funcionan correctamente y presentan una eficiencia aceptable. Puede existir duplicación menor de lógica o respuestas con información adicional no crítica.	Las consultas funcionan parcialmente, pero son poco eficientes, repetitivas o no consideran adecuadamente filtros, relaciones o manejo de errores.	Las consultas fallan, son inseguras, generan respuestas incorrectas o impiden el funcionamiento estable de la aplicación.
EF5. Integración con servicio externo o API de terceros	10 pts	Integra correctamente un servicio externo pertinente al problema. La integración se justifica, se documenta, maneja errores, claves o variables de entorno y aporta valor real a la aplicación.	Integra un servicio externo funcional y relacionado con el proyecto, aunque con documentación limitada o manejo de errores parcial.	La integración es superficial, poco justificada o funciona de manera limitada. Puede depender de datos estáticos o no aportar valor claro al flujo principal.	No existe integración con servicio externo o la integración no funciona.

Criterio	Puntaje	Excelente	Bueno	Básico	Insuficiente
EF6. Despliegue local mediante Docker y docker-compose	15 pts	Incluye Dockerfile para frontend y backend, docker-compose para orquestar frontend, API y base de datos, variables de entorno, instrucciones claras y evidencia de despliegue local exitoso.	Incluye configuración Docker funcional para la mayoría de los servicios. Puede requerir ajustes menores o instrucciones adicionales para ejecutar correctamente.	La configuración Docker es parcial. Algunos servicios no se levantan correctamente o requieren pasos manuales no documentados.	No incluye Docker funcional, no existe docker-compose o el proyecto no puede ejecutarse mediante contenedores.
Presentación y defensa del proyecto	15 pts	El equipo presenta con claridad el problema, solución, arquitectura, funcionalidades, decisiones técnicas, pruebas y despliegue. Todos los integrantes participan y responden adecuadamente preguntas técnicas.	La presentación comunica los aspectos principales del proyecto, aunque con menor profundidad en decisiones técnicas, pruebas o despliegue. La mayoría del equipo participa.	La presentación es poco clara o incompleta. Se explican algunas funcionalidades, pero no se justifica adecuadamente la arquitectura ni las decisiones técnicas.	No se presenta el proyecto, la defensa no evidencia dominio técnico o no se logra demostrar el funcionamiento de la aplicación.